

Техническое задание

Веб-приложение «Конфетная фабрика»

(онлайн-монитор командного турнира по решению задач
с автоматическим учётом решений через informatics.msk.ru)

Версия 1.0

3 июля 2026 г.

Содержание

1	Общие сведения	2
1.1	Что делаем	2
1.2	Роли	2
2	Архитектура и технологический стек	3
2.1	Состав	3
2.2	Стек (обязательный)	3
2.3	Каталог данных	3
3	Игровые правила	4
3.1	Структура задач: n уровней, n^2 задач	4
3.2	Параметры игры	4
3.3	Состояния задачи и механики	4
3.4	Жизненный цикл игры	5
4	Модель данных	5
5	Журнал событий и расчёт состояния	6
5.1	Принцип: состояние = функция журнала	6
5.2	Типы событий	6
5.3	Алгоритм расчёта состояния	7
6	Ссылки на задачи информатикса	8

7	Интеграция с informatics (перенос из standings-edu)	8
7.1	Сервисный аккаунт	8
7.2	Авторизация (Moodle-логин)	8
7.3	API посылка	9
7.4	Инкрементальная выгрузка	10
7.5	Опросчик	10
7.6	Кеш опросчика	10
7.7	Аномалии	11
8	Страницы и интерфейс	11
8.1	Карта маршрутов	11
8.2	Публичное табло /g/{gameId}	12
8.3	Страница команды	12
8.4	Автообновление	12
8.5	Админка /admin	13
9	Конфигурация и развёртывание	14
9.1	Параметры запуска	14
9.2	Развёртывание	14
9.3	Логирование	15
10	Нефункциональные требования	15
11	Критерии приёмки	15

1 Общие сведения

1.1 Что делаем

Веб-версия игры-соревнования «Конфетная фабрика». Существующая эталонная реализация — настольное приложение на Lazarus (описано в отдельном ТЗ «Игра-соревнование “Конфетная фабрика”», файл `TZ-candy-factory.pdf`); все игровые правила и формулы наследуются оттуда с обобщениями, явно описанными в разделе 3.

Ключевые отличия веб-версии от настольной:

1. Приложение разворачивается на сервере и доступно через браузер.
2. Есть **админка** по адресу `/admin` с простейшей авторизацией (логин/пароль в файле на диске).
3. Из админки создаются **игры**; несколько игр могут идти **параллельно и независимо**.
4. При создании игры задаются ссылки на задачи с <https://informatics.msk.ru/> или <https://informatics.mccme.ru/> (это зеркала одного сайта) и список команд с их идентификаторами пользователей (аккаунтов) на информатиксе.
5. У каждой команды — **своя страница** (за логином/паролем, заданными в конфигурации игры) с таблицей её задач, где ячейки — ссылки на условия задач. Внутри каждой строки таблицы (строка = уровень сложности) задачи **перемешаны** индивидуально для каждой команды.
6. Есть **публичная страница игры** — аналог старого табло (запасы, скорость, сетка задач с цветами состояний, гистограмма, таймер), но **без ссылок** на задачи. Доступна всем без авторизации.
7. Решения задач подтягиваются **автоматически** с информатикса по аккаунтам команд (механизм заимствуется из проекта <https://github.com/FynjyBath/standings-edu> и описан в разделе 7 настольно подробно, что обращаться к репозиторию не обязательно).
8. Все действия в игре (покупки задач, покупки тестов, решения) хранятся как **журнал событий** (действие, время). Админ может добавлять, редактировать и удалять события (кроме автоматических решений — их можно только отключать), и всё состояние игры детерминированно пересчитывается из журнала. Страницы участников и зрителей подхватывают изменения автоматически.

1.2 Роли

Роль	Возможности
Администратор (ведущий)	Вход в <code>/admin</code> . Создание/настройка/запуск/завершение игр, ведение журнала событий (покупки и т. п.), просмотр аномалий, просмотр любых страниц.
Команда	Вход на свою страницу игры по логину/паролю команды. Видит общее табло и свою таблицу задач со ссылками на условия и статусами. Никаких действий не совершает — только смотрит и решает задачи на информатиксе.
Зритель	Публичная страница игры без авторизации: табло без ссылок на задачи.

Покупки задач и тестов, как и в настольной версии, вводит администратор (команды сообщают о желании купить голосом/чатом). Решения задач администратор не вводит — они приходят автоматически с информатикса.

2 Архитектура и технологический стек

2.1 Состав

Одно серверное приложение (монолит):

- HTTP-сервер: HTML-страницы (серверный рендеринг шаблонами) + JSON-API для автообновления;
- фоновый **опросчик** (poller) информатикса — одна горутина/поток на процесс, обходит аккаунты всех активных игр;
- хранилище: **SQLite** (один файл БД) + файлы конфигурации и кеша в каталоге данных.

2.2 Стек (обязательный)

- Язык — **Go** (1.22+). Причина: клиент информатикса переносится из standings-edu (Go) почти без изменений.
- HTTP — стандартный **net/http**, шаблоны — **html/template**.
- SQLite — драйвер без сго: **modernc.org/sqlite**.
- Фронтенд — обычный HTML/CSS + ванильный JavaScript (периодический **fetch** JSON-состояния). Без сборщиков, без SPA-фреймворков.
- Итог сборки — один статический бинарник; статика и шаблоны встраиваются через **embed**.

2.3 Каталог данных

Путь задаётся флагом/переменной окружения (**-data-dir**, по умолчанию **./data**):

```
data/
  app.db                # SQLite
  credentials/
    admin_credentials.json  # {"login": "...", "password": "..."}
    informatics_credentials.json # сервисный аккаунт информатикса
  cache/
    informatics_state.json  # кеш опросчика (см. 7.6)
  server.log              # текстовый лог приложения
```

Файл **admin_credentials.json** — «самый простой вход»: логин и пароль открытым текстом, читается при каждой попытке входа (правку файла можно делать без рестарта):

```
{ "login": "admin", "password": "change_me" }
```

3 Игровые правила

Правила полностью повторяют настольную версию, но обобщены с трёх уровней сложности до n уровней.

3.1 Структура задач: n уровней, n^2 задач

- В конфигурации игры задаётся число уровней сложности n (целое, $2 \leq n \leq 6$; значение по умолчанию $n = 3$).
- Задаётся ровно n^2 ссылок на задачи информатикса: n строк (строка r — уровень сложности r , нумерация от простого к сложному) по n задач в строке.
- Набор задач **общий для всех команд**. Но порядок задач **внутри каждой строки** для каждой команды свой: при запуске игры для каждой пары (команда, строка) генерируется и сохраняется случайная перестановка n задач этой строки. Перестановка генерируется один раз и далее не меняется (в том числе при рестарте сервера).
- Ячейки таблицы команды нумеруются сквозными номерами $1 \dots n^2$ по строкам: ячейка (r, c) имеет номер $(r - 1) \cdot n + c$. Из-за перемешивания одна и та же задача у разных команд стоит в разных ячейках *одной и той же строки* — зрители и соперники не могут по номеру ячейки понять, какая именно это задача.

3.2 Параметры игры

Все параметры задаются в конфигурации игры. Для каждого уровня $r = 1..n$:

Параметр	Обозначение	Значение по умолчанию ($n = 3$)
Цена задачи	TaskCost _{r}	12 000 / 12 000 / 12 000
Цена теста (подсказки)	TestCost _{r}	3 000 / 7 000 / 10 000
Нагрузка от задачи	Load _{r}	2 / 2 / 2
Бонус к запасам	AmountBonus _{r}	12 000 / 25 000 / 50 000
Бонус к скорости	SpeedBonus _{r}	4 / 7 / 11

Общие параметры игры:

Параметр	Обозначение	По умолчанию
Стартовые запасы	StartAmount	20 000
Стартовая скорость	StartSpeed	15
Длительность, секунд	Duration	5 100 (1 ч 25 мин)

3.3 Состояния задачи и механики

У каждой пары (команда, задача) — состояние: **hidden** (не куплена, серая), **bought** (куплена, жёлтая), **passed** (решена, зелёная) и счётчик купленных тестов. Механики (эффекты применяются в момент времени события, см. раздел 5):

- **Покупка задачи** (**hidden** → **bought**): amount $\mathrel{-=}$ TaskCost _{r} ; speed $\mathrel{-=}$ Load _{r} .
- **Покупка теста** (только у **bought**): amount $\mathrel{-=}$ TestCost _{r} ; счётчик тестов +1. Число тестов не ограничено. Состояние не меняется.

- **Решение задачи** (`bought` $\rightarrow$ `passed`): `amount` $+=$ `AmountBonusr`; `speed` $+=$ `SpeedBonusr` + `Loadr` (нагрузка возвращается, бонус добавляется).
- Потраченное на задачу и тесты при решении **не возвращается**.
- **Производство**: раз в секунду игрового времени `amount` $+=$ `speed` (формально — см. 5.3).
- Побеждает команда с наибольшим `amount` на момент конца игры.

Отдельных операций «отмена» больше нет: ошибочные действия исправляются редактированием журнала событий в админке (раздел 5).

При вводе покупки админка *предупреждает*, если у команды на этот момент недостаточно средств (`amount` < `TaskCostr` / `TestCostr`) или производительности (`speed` < `Loadr`), — тексты предупреждений «Недостаточно средств», «Недостаточно производительности», — но позволяет сохранить событие после явного подтверждения (чек-бокс/повторное нажатие): администратор — окончательный авторитет.

3.4 Жизненный цикл игры

Статус	Как попадает	Что можно менять
<code>draft</code>	создание игры	всё
<code>running</code>	кнопка «Старт» / наступление <code>start_at</code>	события; продление <code>Duration</code> ; пароли коман
<code>finished</code>	<code>now</code> $\geq$ <code>start_at</code> + <code>Duration</code>	события (временем $\leq$ конца игры)
<code>archived</code>	кнопка «В архив»	ничего

Кнопка «Старт» устанавливает `start_at` := `now`; альтернативно админ может заранее задать плановое время старта, и игра перейдёт в `running` автоматически. Переход `running` $\rightarrow$ `finished` происходит сам по времени; статус — вычисляемая функция времени, отдельного «процесса завершения» нет. Состав задач, n , перестановки и параметры уровней после старта **не редактируются**.

Опрос информатикса продолжается 15 минут после конца игры, чтобы подтянуть посылки, отправленные «на флажке» и протестированные с задержкой; решения с временем посылки $\leq$ конца игры засчитываются, позже — попадают в аномалии (см. 7.7).

4 Модель данных

Схема SQLite (типы условные; TEXT-время — RFC3339 UTC):

```
games(id, title, status_archived INT, n INT,
      start_amount INT, start_speed INT, duration_sec INT,
      start_at TEXT NULL, created_at TEXT)

game_levels(game_id, level INT,          -- 1..n
            task_cost INT, test_cost INT, load INT,
            amount_bonus INT, speed_bonus INT,
            PRIMARY KEY(game_id, level))

game_tasks(id, game_id, level INT, ord INT, -- ord: 1..n внутри уровня,
          chapter_id INT,                  -- как ввёл админ
```

```

url TEXT,                                -- нормализованная ссылка
UNIQUE(game_id, chapter_id))

teams(id, game_id, name TEXT,
      informatics_user_id INT,           -- id аккаунта на информатиксе
      login TEXT, password TEXT,        -- вход на страницу команды
      UNIQUE(game_id, login))

team_task_order(team_id, cell INT,       -- cell: 1..n^2
                task_id,                 -- -> game_tasks.id
                PRIMARY KEY(team_id, cell)) -- материализованная перестановка

events(id, game_id, team_id, task_id NULL,
       type TEXT,                        -- 'buy_task' | 'buy_test' | 'solve'
       at TEXT,                          -- момент события, UTC
       source TEXT,                      -- 'manual' | 'auto'
       run_id INT NULL,                  -- id посылки информатикса (для auto)
       enabled INT DEFAULT 1,
       comment TEXT DEFAULT '',
       created_at TEXT, updated_at TEXT,
       UNIQUE(game_id, run_id))          -- дедупликация auto-решений

anomalies(id, game_id, team_id, task_id, run_id INT,
          run_at TEXT, reason TEXT, resolved INT DEFAULT 0,
          created_at TEXT, UNIQUE(game_id, run_id))

```

Примечания:

- Пароли команд хранятся открытым текстом (осознанное упрощение: ценность — нулевая, турнир одноразовый).
- `team_task_order` заполняется при переходе в `running`: для каждой команды и каждого уровня r — случайная перестановка задач уровня, записанная в ячейки $(r-1)n+1 \dots rn$. Генератор — криптографический или `seeded math/rand`; главное — результат сохраняется и стабилен.
- Сессии (админа и команд) можно не хранить в БД: подписанные cookie (HMAC-SHA256 с серверным секретом, генерируемым при первом запуске в `data/secret`) с полями (роль, `game_id`, `team_id`, срок действия 24 ч).

5 Журнал событий и расчёт состояния

5.1 Принцип: состояние = функция журнала

Никакие «текущие запасы/скорости» в БД не хранятся. Всё состояние игры в любой момент t детерминированно вычисляется из конфигурации игры и списка **включённых** (`enabled=1`) событий с $at \leq t$. Поэтому редактирование прошлого (изменение времени события, удаление, добавление задним числом) автоматически и корректно меняет текущие цифры на всех страницах.

5.2 Типы событий

Тип	Источник	Смысл и эффекты
buy_task	manual	Покупка задачи командой. $-\text{TaskCost}_r$ к запасам, $-\text{Load}_r$ к скорости, задача $\rightarrow$ bought .
buy_test	manual	Покупка теста. $-\text{TestCost}_r$ к запасам, счётчик тестов задачи +1.
solve	auto (poller) или manual	Решение задачи. $+\text{AmountBonus}_r$ к запасам, $+(\text{SpeedBonus}_r + \text{Load}_r)$ к скорости, задача $\rightarrow$ passed . Auto-событие ссылается на run_id послышки.

Правила редактирования в админке:

- ручные события (**source=manual**): добавлять, менять время, команду, задачу, тип; удалять физически;
- автоматические (**source=auto**): удалять и менять нельзя (иначе опросчик создаст их заново); можно **отключить** (**enabled=0**) — событие остаётся в списке зачёркнутым и не участвует в расчёте, а уникальность по **run_id** гарантирует, что опросчик не создаст дубль; можно **поправить время** (поле **at**) — связь с **run_id** сохраняется;
- ручное событие **solve** допустимо (страховка на случай недоступности информатика).

5.3 Алгоритм расчёта состояния

Обозначим $t_0 = \text{start_at}$, $t_{\text{end}} = t_0 + \text{Duration}$, $T = \min(\text{now}, t_{\text{end}})$.

1. Берём включённые события игры с $\text{at} \leq T$, сортируем по (**at**, **id**). События с $\text{at} < t_0$ трактуются как произошедшие в t_0 (валидатор их подсвечивает).
2. Для каждой команды ведём (**amount**, **speed**) := (**StartAmount**, **StartSpeed**) и состояния задач.
3. Идём по целым секундам $k = 1, 2, \dots, \lfloor T - t_0 \rfloor$ и по событиям в хронологическом порядке, применяя на каждом шаге сначала все события с $\text{at} \leq t_0 + k$, затем начисление **amount** += **speed** за секунду k . (Реализация обязана быть эквивалентной этому определению, но эффективной: между событиями скорость постоянна, поэтому суммарное начисление за интервал считается умножением, а не циклом.)
4. Эффекты события применяются согласно таблице типов. Событие «не по правилам» (см. валидацию ниже) эффект всё равно применяет — админ отвечает за журнал.

Результат расчёта: для каждой команды — **amount**, **speed**, по каждой ячейке — состояние и число тестов.

Валидация (не блокирует, а подсвечивает в админке список проблем):

- **buy_task** по задаче не в **hidden**; **buy_test** по задаче не в **bought**; **solve** по задаче не в **bought**;
- в какой-то момент **amount** < 0 или **speed** < 0;
- событие раньше t_0 или позже t_{end} .

Расчёт кешируется в памяти процесса: пересчитывать при (а) изменении журнала/конфигурации, (б) не реже раза в секунду для начислений (достаточно пересчитывать лениво при запросе состояния, если с прошлого расчёта прошло ≥ 1 с или изменились данные).

6 Ссылки на задачи информатикса

Админ вводит n^2 ссылок вида:

```
https://informatics.msk.ru/mod/statements/view.php?chapterid=113449
https://informatics.mccme.ru/mod/statements/view.php?chapterid=2958#1
```

Требования к разбору (нормализации):

1. Допустимые хосты: `informatics.msk.ru`, `www.informatics.msk.ru`, `informatics.mccme.ru`, `www.informatics.mccme.ru`. Это зеркала: считаются **эквивалентными**.
2. Путь: `/mod/statements/view.php` (без учёта регистра), параметр запроса `chapterid` — положительное целое. Фрагмент (`#1`) и прочие параметры игнорируются.
3. Идентификатором задачи считается `chapterid` — он же совпадает с `problem.id` в API посылок (см. 7.3).
4. Канонический URL для показа на страницах команд: `https://informatics.msk.ru/mod/statements/view.php?chapterid=...` (базовый хост берётся из `base_url` в `informatics_credentials.json`, чтобы можно было переключить зеркало одним местом).
5. Дубликаты `chapterid` внутри игры запрещены — ошибка формы.
6. Ссылка, не подошедшая под правила, — ошибка формы с указанием конкретной строки.

Идентификатор аккаунта команды — числовой `user_id` на информатиксе (виден в URL профиля: `/user/profile.php?id=<user_id>`). Валидация: положительное целое.

7 Интеграция с informatics (перенос из standings-edu)

Этот раздел — переработанное под нашу задачу описание клиента информатикса из <https://github.com/FynjyBath/standings-edu> (файл `internal/source/informatics.go`). Реализовать нужно именно это поведение; код из репозитория можно брать за основу.

7.1 Сервисный аккаунт

Файл `data/credentials/informatics_credentials.json`:

```
{
  "username": "...",
  "password": "...",
  "base_url": "https://informatics.msk.ru"
}
```

`base_url` опционален (по умолчанию `https://informatics.msk.ru`); поля обрезаются от пробелов; пустые логин/пароль — фатальная ошибка старта опросчика (но не веб-сервера: страницы должны работать, в админке — заметный баннер «опрос информатикса не работает: ...»).

7.2 Авторизация (Moodle-логин)

HTTP-клиент: таймаут 20 с, cookie jar (сессия живёт в куках).

1. GET {base_url}/login/index.php; из HTML извлечь токен регулярным выражением `name="logintoken"\s+value="([~"]+)"`. Токен не найден — ошибка «informatics login token not found».
2. POST на тот же URL, Content-Type: application/x-www-form-urlencoded, заголовок Referer = URL логина; поля формы: anchor=, logintoken, username, password, rememberusername=0.
3. Разбор ответа: если тело содержит logout.php — успех. Если содержит name="logintoken" или форму логина или loginerrors — ошибка «bad credentials or blocked account».

Повторный логин: сессия считается живой 30 минут с последнего успешного логина; логин выполняется заново (а) по истечении этого срока, (б) немедленно при ответе API «Not authorized» (см. ниже), после чего запрос повторяется один раз. Логин защищён мьютексом — параллельные запросы не должны логиниться наперегонки.

7.3 API посылки

Список посылок пользователя, новые сверху, постранично:

```
GET {base_url}/py/problem/0/filter-runs
    ?problem_id=0&user_id={user_id}&count=1000&page={page}
    &from_timestamp=-1&to_timestamp=-1&lang_id=-1
    &status_id=-1&statement_id=0&with_comment=
```

Ответ — JSON:

```
{
  "result": "success" | "error",
  "message": "...",
  "data": [ {
    "id": 123456,           // id посылки, монотонно растёт
    "create_time": "...",  // время посылки
    "ejudge_status": 0,    // вердикт ejudge
    "ejudge_score": 100,   // может быть null/строкой/числом
    "score": 100,          // то же
    "problem": { "id": 113449 } // == chapterid задачи
  }, ... ],
  "metadata": { "page_count": 3, "count": 2456 }
}
```

Обязательные детали обработки:

- HTTP-статус $\neq 200$ — ошибка (в лог, с телом до 1 КБ).
- `result="error"` и `message="Not authorized"` (без учёта регистра) — перелогин и один повтор (7.2); другой `error` — ошибка с текстом `message`.
- `ejudge_score/score` в живых данных бывают числом, строкой с числом, пустой строкой и `null` — парсер обязан принимать все варианты (нечисловое $\Rightarrow$ «нет значения»).
- **Решённой** посылка считается при `ejudge_status` $\in \{0, 8\}$: 0 = OK, 8 = ACCEPTED («Зачтено»). Остальные вердикты для нас — просто попытки, они нигде не учитываются.

- Поле `create_time`: standings-edu его не разбирает, нам оно **нужно** (время события `solve`). Исполнителю необходимо посмотреть на реальный ответ API и определить точный формат (ожидается строка даты-времени по московскому времени или unix-время). Распарсенное время приводится к UTC. Если поле отсутствует/не парсится — взять момент обнаружения посылки опросчиком и записать предупреждение в лог.
- Размер страницы 1000; страницы, начиная со второй, можно качать параллельно (в standings-edu — до 8 воркеров), но для наших объёмов (свежие аккаунты команд) достаточно последовательной загрузки — параллелизм опционален.

7.4 Инкрементальная выгрузка

Для каждого аккаунта (`user_id`) хранится `max_run_id` — наибольший виденный `id` посылки. Алгоритм `FetchNewRuns(user_id)`:

1. Загрузить страницу 1. Идя по посылкам (они отсортированы по убыванию `id`), собирать посылки с `id > max_run_id`; встретив `id ≤ max_run_id` — остановиться (далее всё старое).
2. Если не встретили старых и `page_count > 1` — продолжить со страницы 2 и далее до первой старой посылки или конца.
3. Если `max_run_id` ещё не было (первый опрос аккаунта) — выкачать все страницы.
4. Обновить `max_run_id` и сохранить кеш (см. 7.6).
5. Вернуть новые посылки.

7.5 Опросчик

Один фоновый цикл с периодом `poll_interval` (конфиг, по умолчанию **30 с**):

1. Составить множество аккаунтов: все команды всех игр в статусе `running`, плюс `finished` менее 15 минут назад. Пусто — спать до следующего цикла.
2. Для каждого аккаунта последовательно (пауза ≥ 1 с между аккаунтами, чтобы не долбить сайт): `FetchNewRuns`.
3. Каждую новую **решённую** посылку сматчить: по `problem.id` найти задачу игры (`game_tasks.chapter_id`), по `user_id` — команду. Посылки по чужим задачам — игнорировать молча.
4. Для сматченной посылки со временем t_{run} (из `create_time`):
 - если по паре (команда, задача) уже есть включённое событие `solve` или запись с тем же `run_id` (событие или аномалия) — пропустить;
 - если на момент t_{run} задача в состоянии `bought` и $t_0 \leq t_{run} \leq t_{end}$ — создать событие `solve` (`source=auto`, `at=trun`, `run_id`);
 - иначе (не куплена ещё/уже решена/вне времени игры) — создать запись в `anomalies` с причиной; админ решит вручную (см. 7.7).
5. Ошибка сети/логина на любом шаге: записать в лог, показать баннер в админке (время и текст последней ошибки), **не** терять накопленный кеш, повторить в следующем цикле. Один упавший аккаунт не должен прерывать обход остальных.

7.6 Кеш опросчика

Файл `data/cache/informatics_state.json` (схема — упрощение standings-edu):

```
{
  "version": 1,
  "accounts": {
    "285757": { "max_run_id": 1234567,
                "updated_at": "2026-07-03T12:00:00Z" }
  }
}
```

- Запись — **атомарная**: сериализовать во временный файл в том же каталоге, затем `rename` поверх целевого (как в `standings-edu`).
- При чтении: файла нет — пустой кеш; не парсится или `version` не совпадает — игнорировать и начать с нуля (это лишь ускорение, источник истины — информатикс и БД).
- Доступ под мьютексом.
- Внимание: сброс кеша приводит к повторной выкачке всех посылок, но **не** к дублям событий — дедупликация по `run_id` в БД (уникальный индекс).

7.7 Аномалии

Страница игры в админке показывает нерешённые аномалии — решённые посылки, которые нельзя было применить автоматически:

- `not_bought` — на момент посылки задача не куплена (например, админ ещё не внёс покупку);
- `already_passed` — задача уже решена;
- `out_of_time` — посылка вне интервала игры.

По каждой аномалии доступны действия: «Принять» (создаёт ручное событие `solve` с временем посылки, аномалия помечается решённой; полезно после того, как админ внёс пропущенную покупку) и «Отклонить» (помечается решённой без событий). Пока аномалия не решена, опросчик не создаёт по этой посылке ничего нового (уникальность по `run_id`).

8 Страницы и интерфейс

Все страницы — на русском, вёрстка простая (таблицы/флексy), без брендинга; должна нормально выглядеть на проекторе (публичная страница) и на ноутбуке/телефоне (страница команды).

8.1 Карта маршрутов

Маршрут	Доступ	Содержимое
GET /	все	Список неархивных игр со ссылками на публичные страницы.
GET /g/{gameId}	все	Публичное табло игры.
GET /g/{gameId}/team	форма входа	Логин команды; после входа — редирект на страницу команды.

GET /g/{gameId}/team/{teamId}	команда или админ	Страница команды.
GET /api/g/{gameId}/state	все	JSON состояния (без ссылок).
GET /api/g/{gameId}/team/{teamId}/state	команда или админ	JSON состояния + ссылки и порядок задач команды.
GET, POST /admin и /admin/...	админ	Админка (см. 8.5).

gameId, teamId — целые id из БД. Неизвестный id — 404. Страница команды под чужой командной сессией — 403.

8.2 Публичное табло /g/{gameId}

Воспроизводит настольный монитор:

1. Заголовок: название игры; крупный таймер «Оставшееся время: ЧЧ:ММ:СС» (до старта — «До начала: ЧЧ:ММ:СС» или «Игра не началась»; после конца — «Игра завершена»).
2. По колонке на команду: имя; «Запасы: N»; «Скорость: N»; сетка $n \times n$ ячеек с номерами $1..n^2$, раскрашенных по состоянию (серый/жёлтый/зелёный; на жёлтой ячейке — бейдж с числом купленных тестов, если > 0). **Без ссылок и без названий задач.**
3. Гистограмма запасов: горизонтальные или вертикальные полосы, длина пропорциональна запасам; масштаб — по максимуму среди команд (лидер = 100%).
4. Индикатор скорости у каждой команды: «бегущая» полоска (CSS-анимация, период обратно пропорционален скорости) — чисто декоративный, как в оригинале.
5. Таблица параметров игры: строки — уровни, столбцы — «Цена задачи», «Цена подсказки», «Нагрузка от задачи», «Бонус запасы», «Бонус скорость».
6. Легенда цветов: «Не куплена» / «Куплена» / «Решена».

8.3 Страница команды

Вход: форма (логин, пароль команды), проверка по `teams` данной игры, cookie на 24 ч. Содержимое — всё то же публичное табло **плюс** собственная таблица задач $n \times n$:

- ячейка — номер + **ссылка** на условие задачи на информатиксе (канонический URL, `target="_blank"`), порядок ячеек — по `team_task_order` этой команды;
- цвет фона ячейки — состояние задачи; бейдж числа купленных тестов;
- подписи строк: «Уровень 1» ... «Уровень n » (при $n = 3$ можно Easy/Middle/Hard);
- ссылки видны и до покупки задачи (состояние `hidden`) — команда должна видеть условия, чтобы выбрать, что купить.

Чужие таблицы задач (и чужой порядок) команде не показываются никогда.

8.4 Автообновление

- Страницы табло и команды раз в **3 секунды** (конфиг) запрашивают свой `/api/.../state` и перерисовывают числа, цвета ячеек, гистограмму без перезагрузки страницы.
- Таймер тикает на клиенте каждую секунду; ответ API содержит `server_time`, по нему клиент корректирует смещение своих часов.

- Ошибка запроса — молча повторить в следующий цикл; после 3 подряд ошибок показать ненавязчивый индикатор «нет связи».
- Обычная перезагрузка страницы (F5) в любом случае даёт актуальное состояние — состояние всегда считается на сервере из журнала.

Формат GET /api/g/{id}/state (командный вариант дополнительно содержит url в ячейках и порядок своей таблицы):

```
{
  "server_time": "2026-07-03T12:00:00Z",
  "status": "running",           // draft|running|finished|archived
  "start_at": "...", "end_at": "...",
  "n": 3,
  "levels": [ {"task_cost":12000, "test_cost":3000, "load":2,
               "amount_bonus":12000, "speed_bonus":4}, ... ],
  "teams": [ {
    "id": 1, "name": "Команда №1",
    "amount": 31337, "speed": 19,
    "cells": [ {"cell":1, "state":"bought", "tests":2}, ... ]
  }, ... ]
}
```

8.5 Админка /admin

Вход: форма логин/пароль, сверка с admin_credentials.json, cookie-сессия 24 ч. Все прочие /admin/* без сессии — редирект на форму входа.

Список игр. Таблица: название, статус, n , число команд, старт, оставшееся время; кнопки «Открыть», «В архив». Кнопка «Создать игру».

Создание/редактирование игры (draft). Форма:

- название; n ; параметры уровней (таблица n строк, предзаполнена дефолтами из 3); StartAmount, StartSpeed, Duration;
- задачи: n полей-«текстареив» (по уровню), в каждый вставляется n ссылок по одной в строке; при сохранении — нормализация и валидация по разделу 6 с точными сообщениями об ошибках;
- команды (динамический список, минимум 2): имя, informatics_user_id, логин, пароль страницы команды; логины уникальны в игре;
- плановое время старта (опционально) и кнопка «Старт сейчас».

При старте: фиксируются перестановки, игра исчезает из редактирования конфигурации.

Страница игры (основной рабочий экран ведущего).

1. Шапка: статус, таймер, кнопки «+15 минут» (продление Duration), «В архив»; баннер ошибок опросчика (последняя ошибка и когда).
2. **Живое табло** — как публичное, но каждая ячейка кликабельна: клик открывает меню ячейки с кнопками «Купить задачу» / «Купить тест» (активность кнопок по текущему состоянию ячейки, как в настольной версии). Кнопка создаёт ручное

событие с `at = now`; при нехватке средств/скорости — предупреждение с подтверждением (см. 3). Здесь же видно, какая informatics-задача стоит в ячейке (подпись с `chapterid` и ссылкой) — админу можно.

3. **Журнал событий**: таблица всех событий игры (время, команда, ячейка/задача, тип, источник, `gun_id`, вкл/выкл, комментарий), сортировка по времени, фильтры по команде и типу. Действия по строке: «Изменить» (форма: время с точностью до секунды, команда, задача, тип — для manual; только время — для auto), «Удалить» (manual), «Отключить»/«Включить» (auto). Кнопка «Добавить событие» — та же форма с ручным временем. После любого изменения — мгновенный пересчёт и перерисовка табло.
4. Панель **предупреждений валидации** (раздел 5.3): список проблемных событий со ссылками на строки журнала.
5. Секция **аномалий** (раздел 7.7).
6. Секция **команды**: логины/пароли страниц команд (с прямыми ссылками), возможность сменить пароль после старта.

Мутации админки — обычные POST-формы или fetch-запросы; CSRF-токен в формах (генерируется в сессии). Одновременная работа двух админов не поддерживается специально, но не должна ломать данные (все мутации — атомарные транзакции SQLite).

9 Конфигурация и развёртывание

9.1 Параметры запуска

Флаги/переменные окружения:

Параметр	По умолчанию	Смысл
<code>-listen</code>	<code>:8080</code>	адрес HTTP-сервера
<code>-data-dir</code>	<code>./data</code>	каталог данных
<code>-poll-interval</code>	<code>30s</code>	период опроса информатикса
<code>-page-refresh</code>	<code>3s</code>	период автообновления страниц (отдаётся клиенту)

9.2 Развёртывание

- Поставка: один бинарник (linux/amd64) + пример `data/`-каталога + unit-файл `systemd` (в комплекте, `Restart=always`).
- HTTPS — забота reverse-proxy (nginx/caddy) перед приложением; приложение слушает обычный HTTP. В README — пример конфига nginx.
- Все времена хранятся и передаются в UTC; отображение на страницах — в часовом поясе сервера (по умолчанию Europe/Moscow — указать в README про переменную TZ).
- При рестарте сервера состояние полностью восстанавливается из БД и кеша; идущие игры продолжаются без потерь (время старта в БД).

9.3 Логирование

Текстовый лог (`data/server.log + stdout`): старты/завершения игр, каждое созданное/изменённое/удалённое событие (кто: `admin/poller`), каждая аномалия, ошибки опросчика, входы в админку и на страницы команд (успех и провал). Формат строки: время, уровень, сообщение.

10 Нефункциональные требования

1. Нагрузка: до 10 параллельных игр, до 8 команд в игре, до 200 одновременных зрителей. С учётом обновления раз в 3 с — 70 RPS лёгких JSON-запросов; ответ из кеша состояния, без пересчёта на каждый запрос.
2. Целостность: журнал событий — единственный источник истины; никакие операции не должны приводить к его частичной записи (транзакции).
3. Опросчик не должен положить информатикс: не более одного запроса в секунду суммарно, инкрементальная выгрузка обязательна.
4. Секреты (пароль сервисного аккаунта информатикса) не должны попадать в логи и в HTML-страницы.
5. Код — с юнит-тестами как минимум на: нормализацию ссылок (6), расчёт состояния из журнала (5.3, включая события задним числом и отключённые), инкрементальную логику `max_run_id`, матчинг посылок и генерацию аномалий.

11 Критерии приёмки

1. **Развёртывание.** На чистом сервере по README: бинарник + каталог данных + `systemd`; админка открывается, вход по паре из `admin_credentials.json`; неправильный пароль — отказ.
2. **Создание игры.** Игра с $n = 3$, 9 ссылками (в т. ч. часть с `informatics.mscme.ru` и с #1 на конце — принимаются и нормализуются), 3 командами. Ошибочная ссылка и дубль задачи — внятные ошибки формы.
3. **Перемешивание.** После старта таблицы двух команд содержат одинаковые множества задач в каждой строке, но в разном порядке (проверить несколько игр); порядок стабилен после рестарта сервера.
4. **Табло.** Публичная страница: запасы растут на `StartSpeed` в секунду, таймер убывает, ссылок на задачи нет в HTML вообще (в том числе в JSON публичного API). Страница команды — те же цифры плюс кликабельные ссылки на условия.
5. **Покупка.** Через админ-табло куплена задача уровня 1: у команды —12 000 запасов, —2 скорости, ячейка жёлтая на всех страницах в течение 3–5 секунд без перезагрузки. Покупка при нехватке средств — предупреждение, сохранение только после подтверждения.
6. **Автозачёт.** Команда сдаёт задачу на информатиксе (вердикт ОК или «Зачтено»): в течение периода опроса появляется событие `solve` с временем посылки; запасы и скорость меняются по правилам; ячейка зелёная. Посылка по некупленной задаче — попадает в аномалии, после «Принять» (и внесения покупки задним числом) зачитывается.

7. **Редактирование истории.** Смещение времени покупки на 10 минут назад увеличивает текущие запасы ровно на $10 \cdot 60 \cdot \text{Load}_r$ (недоначисленная скорость) — сверить с ручным расчётом. Удалённое событие исчезает со всех страниц после очередного автообновления. Отключённое auto-решение не возвращается опросчиком.
8. **Параллельность.** Две одновременно идущие игры не влияют друг на друга (в т. ч. одна и та же задача информатикса в двух играх и разные команды одного «владельца»).
9. **Отказоустойчивость.** При недоступном информатиксе табло и админка полностью работоспособны, в админке — баннер с ошибкой; после восстановления связи пропущенные решения подтягиваются с корректным временем отправки. После `kill -9` и рестарта — игра продолжается, дублей решений нет.
10. **Финиш.** По истечении времени начисление останавливается, статус `finished`; отправка, отправленная за минуту до конца, но протестированная после, — зачтена автоматически (окно 15 минут).